

CHAPTER 04

OpenCV 주요 기능

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

4장 OpenCV 주요 기능

4.1 카메라와 동영상 파일 다루기

4.2 다양한 그리기 함수

4.3 이벤트 처리

4.4 OpenCV 데이터 파일 입출력

4.5 유용한 OpenCV 기능

4.3 이벤트 처리

이벤트 처리

- 이벤트(event) : 마우스, 키보드 등 하드웨어를 동작 시켰을 때 각 장치에서 운영체제에 알리는 신호
- 이벤트 핸들러(event handler) : 이벤트가 발생했을 때 처리해야 할 일을 작성해놓은 함수, 운영체제에 의해 자동으로 호출됨(콜백 함수)



키보드 이벤트 처리

```
int waitKey(int delay = 0);
```

- **delay** 키 입력을 기다릴 시간(밀리초 단위). $\text{delay} \leq 0$ 이면 무한히 기다립니다.
- **반환값** 눌려진 키 값. 지정한 시간 동안 키가 눌리지 않았으면 -1을 반환합니다.

- 영상 윈도우에서 발생하는 키보드 이벤트를 처리해 주는 함수
- waitKey 함수는 키보드 입력시 아스키코드를 리턴하거나 프로그램의 실행을 지연시키고 또는 imshow로 출력한 영상을 화면에 업데이트하는 3가지 기능을 동시에 수행
- 영상 윈도우를 선택한 상태에서 키보드를 입력해야 한다. 콘솔창에서는 입력을 받을 수 없다.

키보드 이벤트 처리

- 먼저 waitKey() 함수를 사용하여 어떤 키가 눌렸는지 체크하고 각 경우에 처리할 코드를 실행

```
while (true) {  
    int keycode = waitKey(10);  
  
    if (keycode == 'a')  
        // 처리코드  
    else if (keycode == 'b')  
        // 처리코드  
    else if (keycode == 'c')  
        // 처리코드  
    else if (keycode == 'd')  
        // 처리코드  
}
```

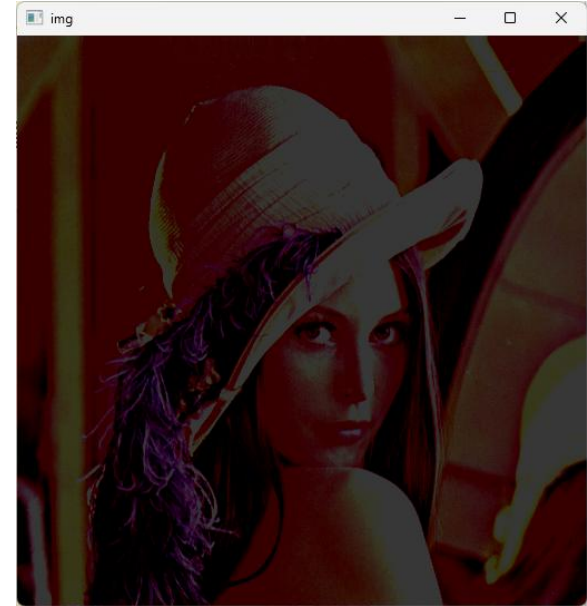
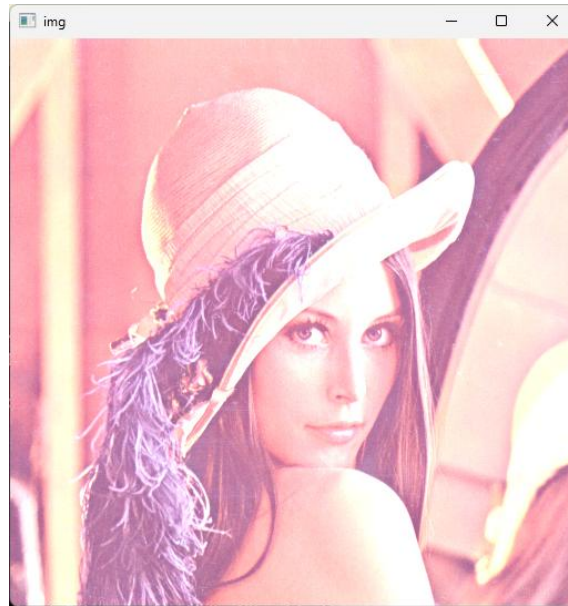
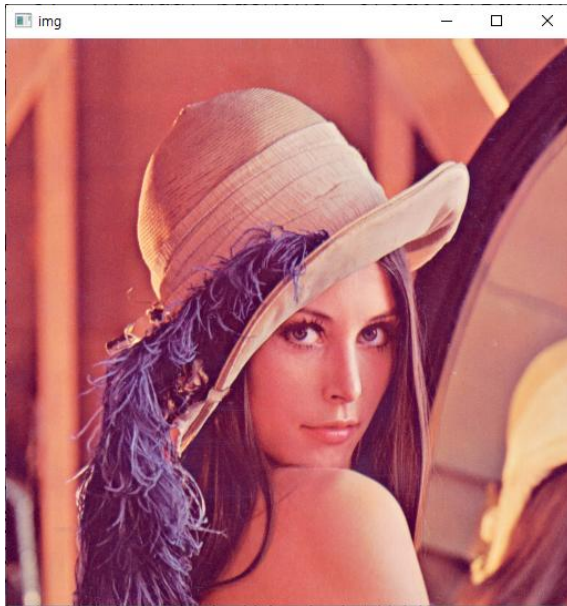
코드4-10 : 키보드 이벤트 처리

```
#include "opencv2/opencv.hpp"
#include <iostream>
using namespace cv;
using namespace std;
int main(void)
{
    Mat img = imread("lenna.bmp");
    if (img.empty()) { cerr << "Image load failed!" << endl; return -1; }
    imshow("img", img);
    int keycode;

    while (true) {
        imshow("img", img);
        keycode = waitKey();
        if (keycode == 'b') img += Scalar(100,100,100);
        else if (keycode == 'd') img -= Scalar(100,100,100);
        else if (keycode == 'q' || keycode == 'Q') break;
    }
    return 0;
}
```

코드4-10 : 키보드 이벤트 처리

- 주의 : waitKey 함수는 콘솔창을 선택한 상태에서는 키보드 입력을 받을 수 없다.



마우스 이벤트 처리

- OpenCV는 영상 출력 창에서 발생하는 마우스 이벤트를 사용자에게 전달하는 기능을 제공함
- 영상 출력 창에서 마우스를 클릭하여 메뉴를 선택하거나 마우스를 드래그하여 영상에 그림을 그리는 등의 동작을 수행할 수 있음
- 마우스 이벤트를 처리하려면 먼저 마우스 이벤트 핸들러(콜백 함수)를 작성해야 함
- 마우스 이벤트 발생시 운영체제가 호출할 마우스 이벤트 핸들러(콜백함수)를 등록해야 함



마우스 이벤트 처리

```
void setMouseCallback(const String& winname, MouseCallback onMouse,
                     void* userdata = 0);
```

- `winname` 마우스 이벤트를 처리할 창의 이름
- `onMouse` 마우스 이벤트를 위한 콜백 함수 이름
- `userdata` 콜백 함수에 전달할 사용자 데이터의 포인터

- 특정 창에서 발생하는 마우스 이벤트를 처리해주는 마우스 콜백 함수(이벤트 핸들러)를 등록할 때 사용함
- 콜백 함수의 자료형 `MouseCallback`은 다음처럼 타입 재정의되어 있음

```
typedef void (*MouseCallback)(int event, int x, int y, int flags, void* userdata);
-> typedef void (*    )(int event, int x, int y, int flags, void* userdata) MouseCallback
```

- `onMouse`는 함수 포인터이므로 마우스 콜백함수의 이름을 `onMouse`에 전달해야 함

마우스 이벤트 처리

```
void event_handler(int event, int x, int y, int flags, void* userdata)
{
    // 내용은 사용자가 작성
}
```

- 마우스 콜백 함수 이름은 사용자 마음대로 지정해도 되지만, 함수의 매개변수 목록과 반환형은 반드시 정해진 형태를 따라야 함
- **event** : MouseEventTypes 열거형 상수 중 하나가 전달됨
- **x, y** : 마우스 이벤트가 발생한 위치의 x 좌표와 y 좌표가 전달됨
- **flags** : 마우스 이벤트 발생시 마우스 또는 키보드의 상태 정보를 전달, flags인자에는 MouseEventFlags 열거형 상수의 논리합조합이 전달됨
- **userdata** : setMouseCallback() 함수에서 설정한 사용자 데이터의 주소가 전달됨
- setMouseCallback() 호출시 userdata를 설정하지 않았다면 마우스 콜백 함수의 userdata 인자에는 항상 0(NULL)이 전달됨

마우스 이벤트 처리

표 4-8 MouseEventTypes 열거형 상수

MouseEventTypes 열거형 상수	값	설명
EVENT_MOUSEMOVE	0	마우스가 창 위에서 움직이는 경우
EVENT_LBUTTONDOWN	1	마우스 왼쪽 버튼을 누른 경우
EVENT_RBUTTONDOWN	2	마우스 오른쪽 버튼을 누른 경우
EVENT_MBUTTONDOWN	3	마우스 가운데 버튼을 누른 경우
EVENT_LBUTTONUP	4	마우스 왼쪽 버튼을 떼는 경우
EVENT_RBUTTONUP	5	마우스 오른쪽 버튼을 떼는 경우
EVENT_MBUTTONUP	6	마우스 가운데 버튼을 떼는 경우
EVENT_LBUTTONDBLCLK	7	마우스 왼쪽 버튼을 더블클릭하는 경우
EVENT_RBUTTONDBLCLK	8	마우스 오른쪽 버튼을 더블클릭하는 경우
EVENT_MBUTTONDBLCLK	9	마우스 가운데 버튼을 더블클릭하는 경우
EVENT_MOUSEWHEEL	10	마우스 휠을 앞뒤로 돌리는 경우
EVENT_MOUSEHWHEEL	11	마우스 휠을 좌우로 움직이는 경우

마우스 이벤트 처리

표 4-9 MouseEventFlags 열거형 상수

MouseEventFlags 열거형 상수	값	설명
EVENT_FLAG_LBUTTON	1	마우스 왼쪽 버튼이 눌러 있음
EVENT_FLAG_RBUTTON	2	마우스 오른쪽 버튼이 눌러 있음
EVENT_FLAG_MBUTTON	4	마우스 가운데 버튼이 눌러 있음
EVENT_FLAG_CTRLKEY	8	 키가 눌러 있음
EVENT_FLAG_SHIFTKEY	16	 키가 눌러 있음
EVENT_FLAG_ALTKEY	32	 키가 눌러 있음

예제1: 마우스 이벤트 처리

- Lenna 영상에서 마우스를 클릭하면 클릭한 버튼과 좌표를 콘솔창에 출력해주는 프로그램을 작성하시오.
- Q를 누르면 종료하도록 하고 같은 코드를 반복하지 말것

```

D:\Users\2sungryul\Dropbox\Lecture\2021년2학...
좌측버튼: [271, 148]
좌측버튼: [271, 148]
좌측버튼: [368, 184]
좌측버튼: [197, 216]
좌측버튼: [339, 294]
우측버튼: [158, 270]
우측버튼: [170, 135]
우측버튼: [369, 105]
우측버튼: [403, 292]
가운데버튼: [319, 181]
가운데버튼: [318, 187]
가운데버튼: [143, 140]
가운데버튼: [277, 307]
가운데버튼: [366, 320]

```



예제 1: 마우스 이벤트 처리

```
#include <iostream>
#include <opencv2/opencv.hpp>
using namespace std;
using namespace cv;
void on_mouse(int event, int x, int y, int flags, void* userdata);
int main(void)
{
    Mat img = imread("lenna.bmp");
    if (img.empty()) { cerr << "Image load failed!" << endl; return -1; }

    namedWindow("img");
    setMouseCallback("img", on_mouse);
    imshow("img", img);
    waitKey();

    return 0;
}
```

예제1: 마우스 이벤트 처리

```
void on_mouse(int event, int x, int y, int flags, void* userdata)
{
    // 함수내부에서 모든 매개변수를 사용할 필요는 없다. flags, userdata는 미사용
    switch (event) {
        case EVENT_LBUTTONDOWN:
            cout << "좌측버튼: " << Point(x, y) << endl;
            break;
        case EVENT_RBUTTONDOWN:
            cout << "우측버튼: " << Point(x, y) << endl;
            break;
        case EVENT_MBUTTONDOWN:
            cout << "가운데버튼: " << Point(x, y) << endl;
            break;
        default:
            break;
    }
}
```


예제1: 마우스 이벤트 처리

- 마우스 이벤트 발생시 실행 흐름을 이해하는 것이 매우 중요

```
// 메인함수
int main(void)
{
    ...
    namedWindow("img");
    setMouseCallback("img", on_mouse);
    imshow("img", img);
    waitKey();
    return 0;
}
```

이벤트 발생시

```
// 이벤트 핸들러
void on_mouse(int event, int x, int y, int flags,
void* userdata)
{
    ...
}
```

이벤트 핸들러 종료시

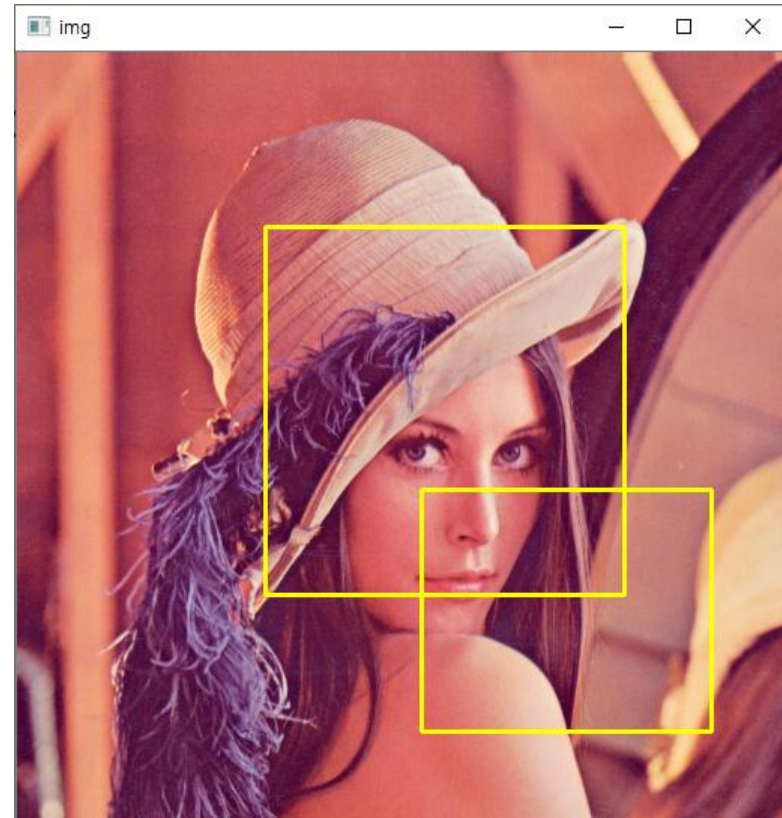
예제2: 마우스 이벤트 처리

- Lenna 영상에서 마우스 왼쪽버튼을 누른상태로 드래킹한 후 버튼을 떼면 누를때 좌표와 땔때 좌표를 이용하여 영상에 사각형을 출력해주는 프로그램을 작성하시오. 또한 이벤트명과 좌표를 출력하라.
- Q를 누르면 종료하도록 하고 같은 코드를 반복하지 말 것

```

D:\Users\2sungryul\Dropbox\Lecture\2020년2학기\프로...
EVENT_LBUTTONDOWN: 165, 116
EVENT_LBUTTONUP: 404, 361
EVENT_LBUTTONDOWN: 269, 291
EVENT_LBUTTONUP: 462, 452

```



예제2: 마우스 이벤트 처리

```
#include <iostream>
#include <opencv2/opencv.hpp>
using namespace std;
using namespace cv;
Mat img;
void on_mouse(int event, int x, int y, int flags, void* userdata);
int main(void)
{
    img = imread("lenna.bmp");
    if (img.empty()) { cerr << "Image load failed!" << endl; return -1; }
    namedWindow("img");
    setMouseCallback("img", on_mouse);
    int key;
    while (true) {
        imshow("img", img);
        key = waitKey(50);
        if (key == 'q') break;
    }
    return 0;
}
```

예제2: 마우스 이벤트 처리

```
void on_mouse(int event, int x, int y, int flags, void* userdata)
{
    // 함수내부에서 모든 매개변수를 사용할 필요는 없다. flags, userdata는 미사용
    static Point pt1;
    switch (event) {
    case EVENT_LBUTTONDOWN:
        pt1 = Point(x, y);
        cout << "EVENT_LBUTTONDOWN: " << x << ", " << y << endl;
        break;
    case EVENT_LBUTTONUP:
        cout << "EVENT_LBUTTONUP: " << x << ", " << y << endl;
        rectangle(img, pt1, Point(x, y), Scalar(0, 255, 255), 2);
        break;
    default:
        break;
    }
}
```

예제2: 마우스 이벤트 처리

- 마우스 이벤트 발생시 실행 흐름을 이해하는 것이 매우 중요

```
// 메인함수  
int main(void)  
{  
    ...  
    while (true) {  
        ...  
    }  
}
```

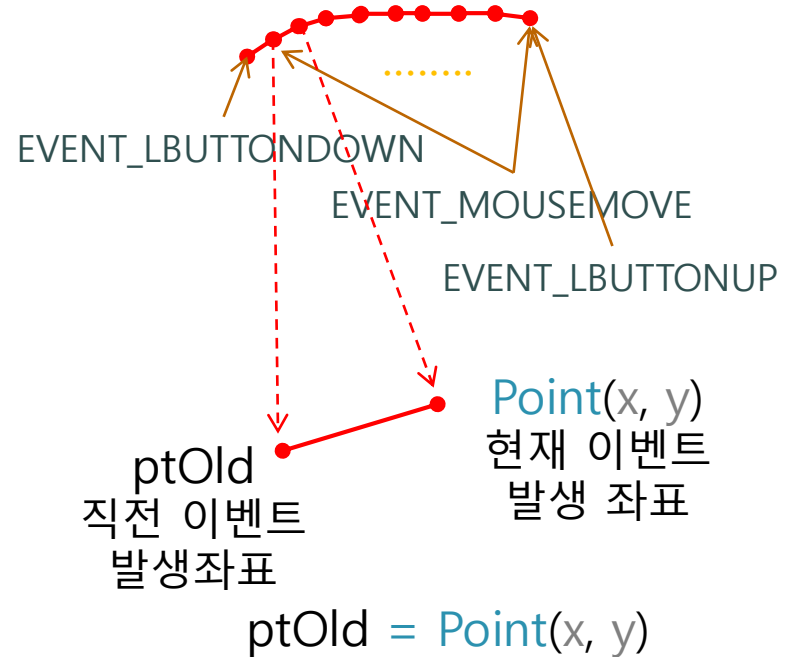
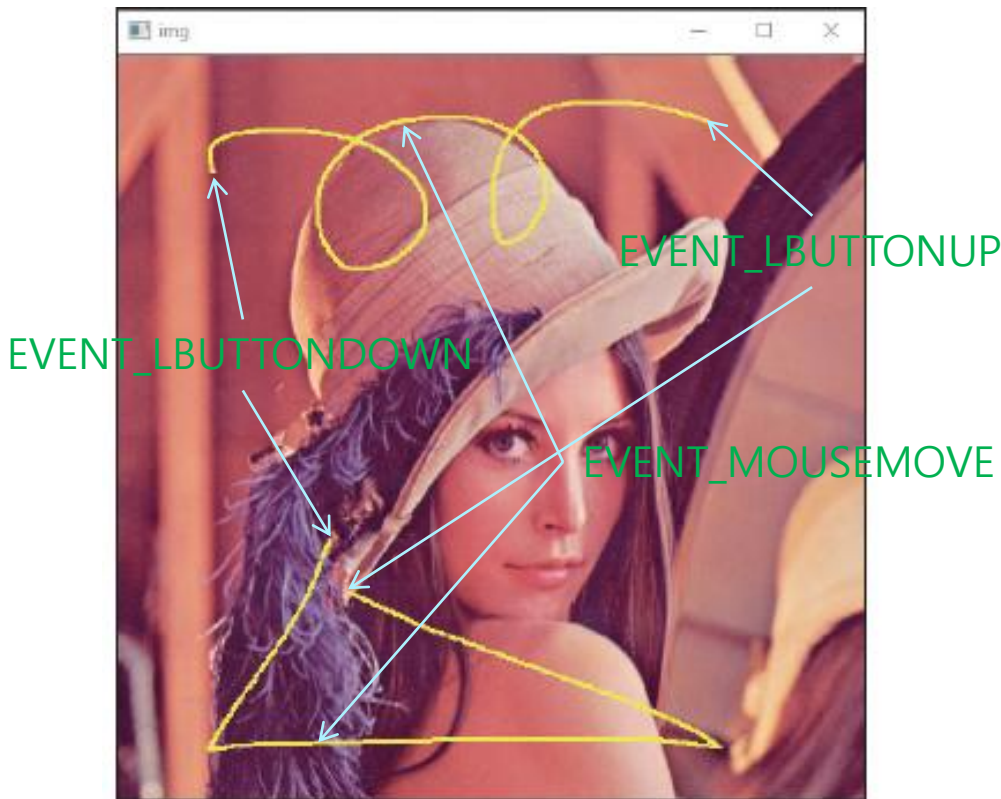
이벤트 발생시

```
// 이벤트 핸들러  
void on_mouse(int event, int x, int y, ... )  
{  
    ...  
}
```

이벤트 핸들러 종료시

코드4-11 : 그림 그리기

- 마우스를 드래킹할 때 EVENT_MOUSEMOVE 이벤트가 발생하고 이때 이전 이벤트 발생 좌표와 현재 이벤트 발생좌표를 잇는 직선을 그려줌



코드4-11 : 그림 그리기

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
void event_handler(int event, int x, int y, int flags, void* userdata);
Mat img; // main함수와 이벤트 핸들러가 변수를 공유
int main(void)
{
    img = imread("lenna.bmp");
    if (img.empty()) { cerr << "Image load failed!" << endl; return -1;}

    namedWindow("img");
    setMouseCallback("img", event_handler);
    int key;
    while (true) {
        imshow("img", img);
        key = waitKey(10);
        if (key == 'q') break;
    }
    return 0;
}
```

코드4-11 : 그림 그리기

```
void event_handler(int event, int x, int y, int flags, void* userdata)
{
    static Point ptOld;    // 이벤트 핸들러 종료후에도 값을 유지해야 함
    switch (event) {
        case EVENT_LBUTTONDOWN:    // 왼쪽 버튼 누를때 이벤트
            ptOld = Point(x, y);
            cout << "EVENT_LBUTTONDOWN: " << x << ", " << y << endl;
            break;
        case EVENT_LBUTTONUP:    // 왼쪽 버튼 떼때 이벤트
            cout << "EVENT_LBUTTONUP: " << x << ", " << y << endl;
            break;
        case EVENT_MOUSEMOVE:    // 드래깅 이벤트
            if (flags & EVENT_FLAG_LBUTTON) {
                line(img, ptOld, Point(x, y), Scalar(0, 255, 255), 2);
                ptOld = Point(x, y);
            }
            break;
    }
}
```


코드4-11 : 그림 그리기

- 마우스 이벤트 발생시 실행 흐름을 이해하는 것이 매우 중요

```
// 메인함수  
int main(void)  
{  
    ...  
    while (true) {  
        ...  
    }  
}
```

이벤트 발생시

```
// 이벤트 핸들러  
void on_mouse(int event, int x, int y, ... )  
{  
    ...  
}
```

이벤트 핸들러 종료시

코드4-11 : 그림 그리기

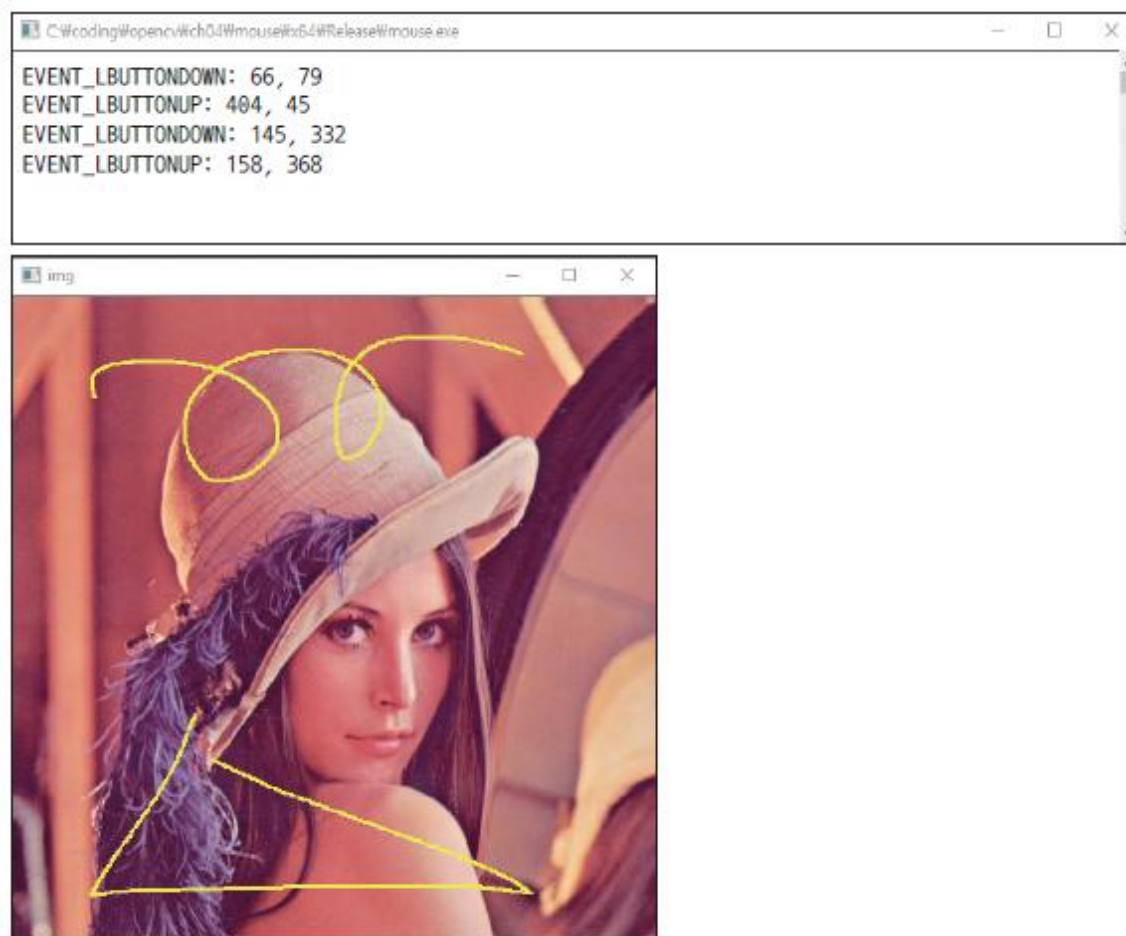


그림 4-8 마우스 이벤트 처리 예제 실행 화면

예제3: 2개 이상의 함수에서 변수 공유하기

- 방법1: 공유할 변수를 전역변수로 선언하고 모든 함수에서 사용
- 방법2: 공유할 변수를 **main함수에 지역변수로 선언하고** 그것의 주소를 이벤트 핸들러의 매개 변수(**void 포인터**)로 전달하는 방법(초기화함수에서 인수로 전달 -> 이벤트 핸들러 호출시 운영체제에 의해 전달됨)

예제3: void 포인터 사용하기

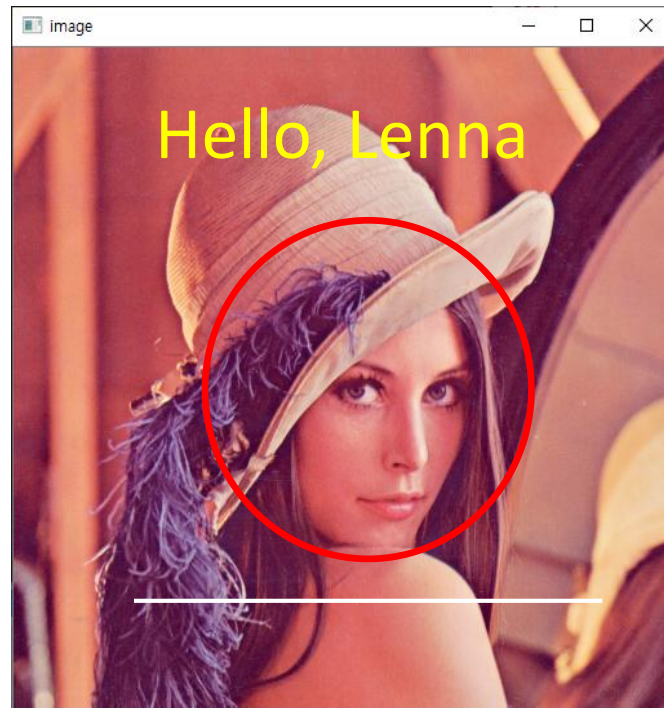
```
#include <iostream>
#include <opencv2/opencv.hpp>
using namespace std;
using namespace cv;
void on_mouse(int event, int x, int y, int flags, void* userdata);
int main(void)
{
    Mat img = imread("lenna.bmp"); // 공유변수를 지역변수로 선언
    if (img.empty()) { cerr << "Image load failed!" << endl; return -1; }
    namedWindow("img");
    setMouseCallback("img", on_mouse, &img); //Mat객체의 주소를 인자전달
    int key;
    while (true) {
        imshow("img", img);
        key = waitKey(50);
        if (key == 'q') break;
    }
    return 0;
}
```

예제3: void 포인터 사용하기

```
void on_mouse(int event, int x, int y, int flags, void* userdata)
{
    // &img(Mat*) -> userdata(void*)
    static Point pt1;
    switch (event) {
    case EVENT_LBUTTONDOWN:
        pt1 = Point(x, y);
        cout << "EVENT_LBUTTONDOWN: " << x << ", " << y << endl;
        break;
    case EVENT_LBUTTONUP:
        cout << "EVENT_LBUTTONUP: " << x << ", " << y << endl;
        rectangle(*(Mat*)userdata, pt1, Point(x, y), Scalar(0, 255, 255), 2);
        // Mat*으로 형변환 후에 간접참조연산 *
        break;
    default:
        break;
    }
}
```

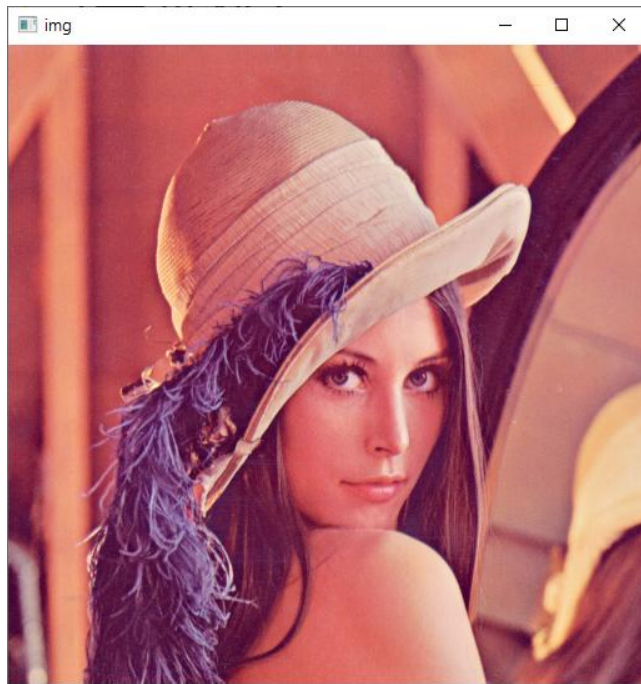
실습과제1

- Lenna 영상을 컬러 영상으로 읽은 후 화면에 출력하고 l(엘)을 누르면 얼굴 아래에 직선을 그리고 c를 누르면 얼굴주위에 원을 그리고 t를 누르면 문자열을 영상 상단에 출력하고 기본 구조는 무한 반복문으로 작성하고 q를 누르면 종료하는 코드를 작성 하시오.



실습과제2

- Lenna 영상 위에서 발생하는 마우스 이벤트의 횟수를 세어서 출력하는 프로그램을 작성하시오. 왼쪽버튼 down, 왼쪽버튼 up, 마우스 move 이벤트만 발생횟수를 발생할 때마다 화면에 다음처럼 출력하라.

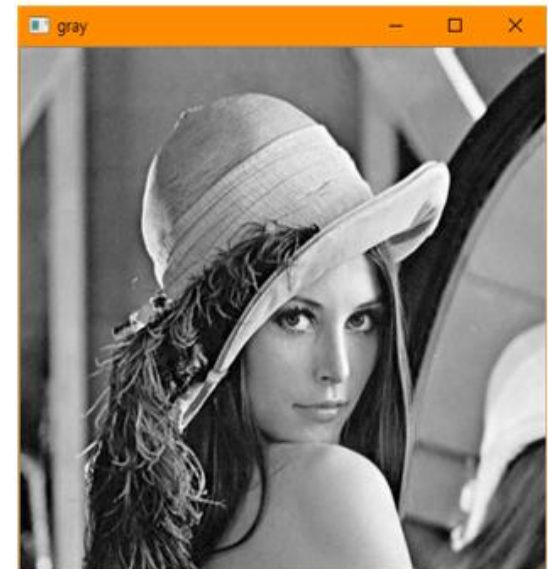


```
D:\Users\W2sungryul\Dropbox\Lecture\2021년2학...
EVENT_MOUSEMOVE: 223
EVENT_LBUTTONDOWN: 12
EVENT_LBUTTONDOWN: 13
EVENT_LBUTTONDOWN: 14
EVENT_MOUSEMOVE: 224
EVENT_LBUTTONDOWN: 13
EVENT_LBUTTONDOWN: 14
EVENT_MOUSEMOVE: 225
EVENT_MOUSEMOVE: 226
EVENT_MOUSEMOVE: 227
EVENT_MOUSEMOVE: 228
EVENT_MOUSEMOVE: 229
EVENT_MOUSEMOVE: 230
EVENT_MOUSEMOVE: 231
EVENT_MOUSEMOVE: 232
```

실습과제3

- Lenna 영상을 그레이스케일 영상으로 변환 후 화면에 출력하고
마우스로 클릭하는 화면 좌표와 그점에서 화소값을 출력하는
프로그램을 작성하시오. 주의: 마우스 좌표계와 Mat::at 함수의
매개변수를 잘 연결할 것

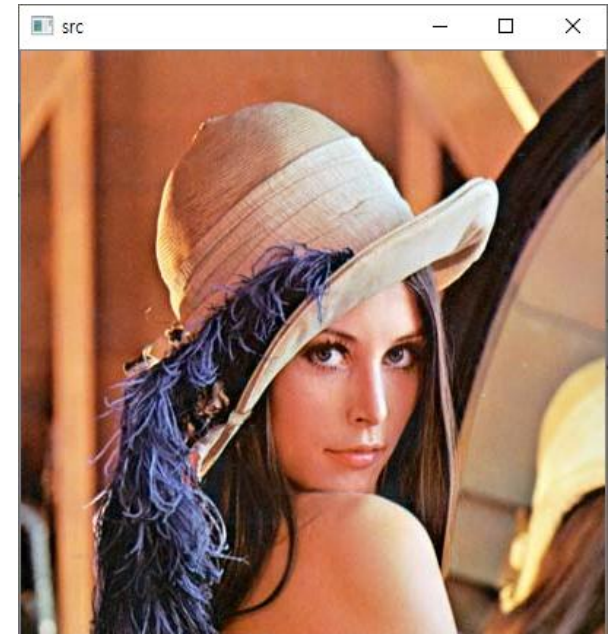
```
D:\Users\2sungryul\Dropbox\Lecture\2019년2학기\프로그램...
좌표: (367,65), 화소값: 23
좌표: (385,272), 화소값: 233
좌표: (21,22), 화소값: 174
좌표: (19,380), 화소값: 176
좌표: (273,376), 화소값: 218
좌표: (216,207), 화소값: 210
좌표: (254,200), 화소값: 167
좌표: (195,97), 화소값: 200
```



실습과제4

- Lenna 영상을 컬러 영상으로 화면에 출력하고 마우스로 클릭하는 화면 좌표와 해당 좌표의 화소값(B,G,R)을 출력하는 프로그램을 작성하시오. 주의: 마우스 좌표계와 Mat::at함수의 매개변수를 잘 연결할 것

```
D:\Users\2sungryul\Dropbox\Lecture\2019년2학기\프로그...
좌표: (395, 19), 화소값(B,G,R): (16, 20, 31)
좌표: (394, 262), 화소값(B,G,R): (198, 235, 239)
좌표: (12, 15), 화소값(B,G,R): (100, 151, 224)
좌표: (21, 395), 화소값(B,G,R): (194, 215, 213)
좌표: (254, 345), 화소값(B,G,R): (166, 199, 232)
좌표: (216, 203), 화소값(B,G,R): (5, 14, 34)
좌표: (185, 94), 화소값(B,G,R): (174, 195, 222)
좌표: (175, 169), 화소값(B,G,R): (164, 121, 130)
좌표: (315, 166), 화소값(B,G,R): (12, 22, 39)
```



실습과제5

- Lenna 영상을 컬러 영상으로 화면에 출력하고 마우스로 드래킹 한 영역을 추출하여 화면에 출력하는 프로그램을 작성 하시오. 코드 4-11을 참고할 것



레나 화면에서 마우스로
드래킹한 영역임

실습과제6

- 크기가 400X200, 배경색이 백색인 컬러영상을 생성하고 마우스 왼쪽 버튼을 누르면 배경색이 Red가 되고 마우스 오른쪽 버튼을 누르면 배경색이 Blue가 되도록 프로그램을 작성하시오.



초기상태



마우스 왼쪽버튼 클릭

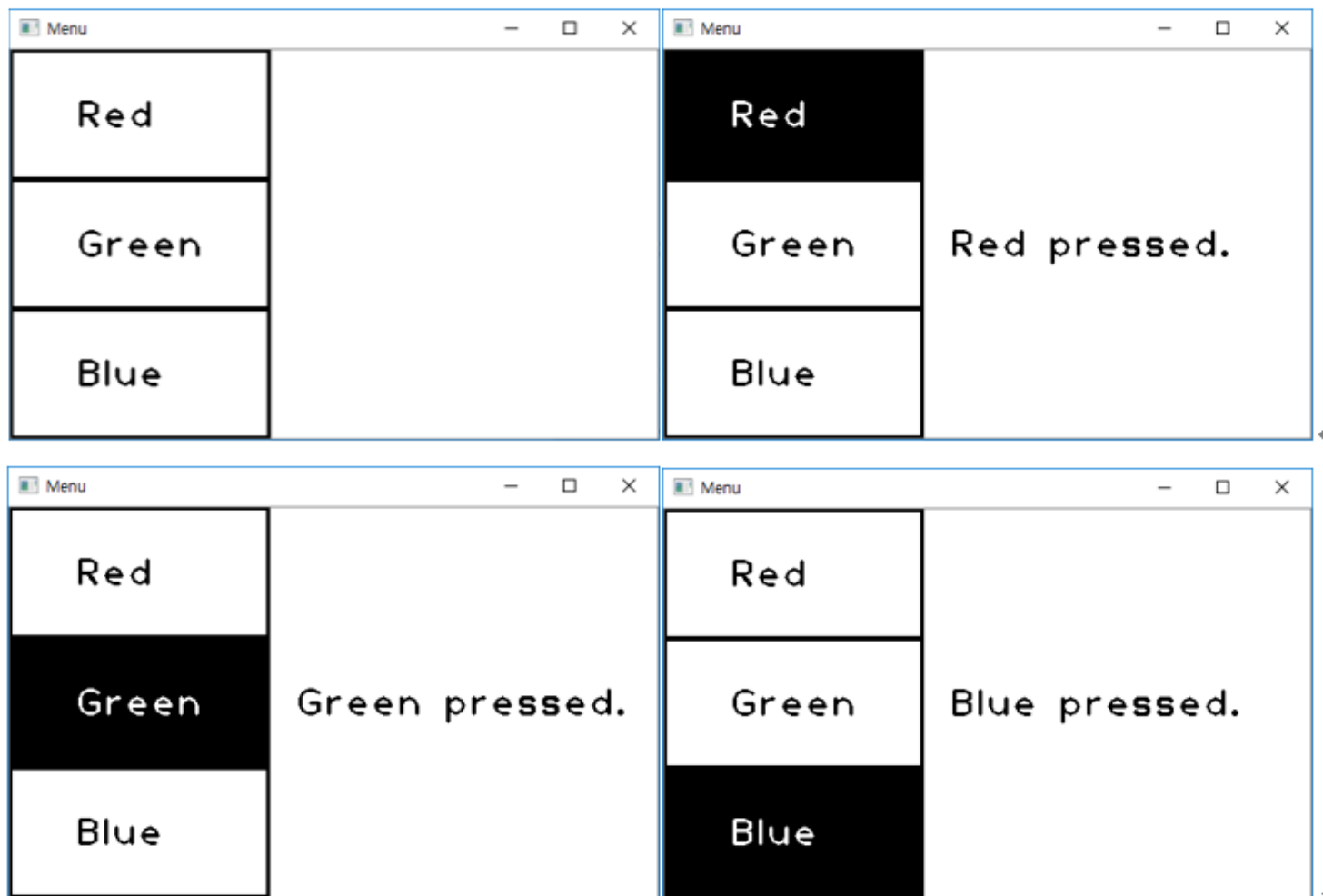


마우스 오른쪽버튼 클릭

실습과제 7

- 다음과 같이 동작하는 프로그램을 작성하시오.
 - ① 초기화면은 좌측상단 그림처럼 작성할 것. 전체화면은 폭 500픽셀, 높이 300픽셀, 메뉴박스의 크기는 폭 200픽셀, 높이 100픽셀, 문자의 폰트와 위치는 적당히 보기 좋게 조정할 것
 - ② Red 사각형 안에서 마우스 좌측버튼을 클릭하면 우측상단 그림처럼 실행되어야 함
 - ③ Green 사각형 안에서 마우스 좌측버튼을 클릭하면 좌측하단 그림처럼 실행되어야 함
 - ④ Blue 사각형 안에서 마우스 좌측버튼을 클릭하면 우측하단 그림처럼 실행되어야 함

실습과제7



과제 제출 방법

- 소스파일, 라인단위 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.